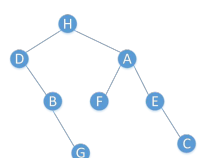


普及组初赛模拟 20

一. 单项选择题 (共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

- 下列选项中, 属于计算机输出设备的是 ()。
A. 鼠标 B. 显示器 C. 键盘 D. CPU
- 4KB 的内存能存储 () 个汉字的机内码。
A. 512 B. 1024 C. 2048 D. 4096
- 一个二进制数的反码为 01011100, 则其原码为 ()。
A. 01011100 B. 10100011 C. 10100100 D. 01011101
- 若 $A = \text{True}$, $B = \text{True}$, $C = \text{False}$, $D = \text{False}$, 则下面表达式值为真是 ()。
A. $(A||B)\&\&!B||C\&\&D$ B. $C\&\&A\&\&(B||!C)||C\&\&D$
C. $B\&\&!B||C||D\&\&!A$ D. $D||(!B||C\&\&!A)||!D\&\&B\&\&!C$
- 小明的 U 盘尚有剩余空间 10GB, 他的电脑硬盘里有 8196 份文档, 850 张照片, 1000 首歌曲, 50 部电影, 每份文档占用的空间是 512KB, 每张照片占用的空间是 5MB, 每首歌曲占用的空间是 10MB, 每部电影占用的空间是 3000MB, 那么下面描述正确的是 ()。
A. 可以把所有文档以及所有照片都拷贝到 U 盘里;
B. 把所有照片拷贝到 U 盘里之后还能再拷贝 2 部电影到 U 盘里;
C. 可以将 5000 份文档, 500 张照片, 300 首歌曲, 1 部电影一起拷贝到 U 盘里。
D. 可以将所有歌曲全部拷贝到 U 盘里, 并且此时 U 盘里没有任何剩余空间。
- 表达式 $(a+b)*c/d-e\%f*g$ 的前缀表达式为 ()。
A. $/*+abcd-*\%efg$ B. $/*+abcd-\%e*fg$
C. $-/*+abcd\%e*fg$ D. $-/*+abcd*\%efg$
- 用数码 0, 1, 2, 3, 4 可以组成 () 个小于 1000 的且没有重复数字的自然数。
A. 25 B. 31 C. 65 D. 69
- 一个由 10 个结点的无向图最少可以有 () 边。
A. 0 B. 1 C. 9 D. 45
- 下图是一棵二叉树, 它的后序遍历是 ()。

A. DBGCEFAH B. FCEAGBDH C. GBDFCEAH D. GBDFECAH

10. 若有以下语句, 则叙述正确的是 ()。

```
int *p[100];
```

```
cout << p << endl;
cout << &p << endl;
cout << &p[0] << endl;
```

- A. 3 行输出都一样 B. 仅 1, 2 行输出一样
C. 仅 1, 3 行输出一样 D. 仅 2, 3 行输出一样

11. 表达式 $15\%6/2 + 14\&5^3$ 的值为 ()。

- A. 4 B. 6 C. 7 D. 8

12. 设电文中出现的字母为 A, B, C, D, E , 每个字母在电文中出现的次数分别为 70, 270, 50, 60 和 110, 按哈夫曼编码, 则字母 C 的编码不可能是 ()。

- A. 01 B. 001 C. 1111 D. 1011

13. 下面代码的时间复杂度为 ()。

```
long long func(long long n) {
    if(n <= 1) return 1;
    return func(n-1) + func(n/2);
}
```

- A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(2^n)$

14. 在 C 程序中如有如下语句: `int a = 0011; printf(“%d”, a);` 则结果是 ()。

- A. 3 B. 9 C. 11 D. 编译出错

15. 有一个循环队列, 队列空间是 50, 当前队首指针 $head = 23$, 指向队首元素的前一个位置, 队尾指针 $tail = 5$, 指向队尾元素, 那么当前队列中共有 () 个元素。

- A. 17 B. 18 C. 31 D. 32

二. 阅读程序 (程序输入不超过数组或字符串定义范围; 判断题正确填 $\sqrt{}$, 错误填 $\times$; 除特殊说明外, 判断题 1.5 分, 选择题 3 分, 共计 40 分)

1.

```
1#include <bits/stdc++.h>
2using namespace std;
3int n, k, e, tmp, f[105], opt[105][2];
4int main()
5{
6    memset(f, -1, sizeof(f));
7    cin >> n >> k;
8    f[0] = 0;
9    for (int i = 1; i <= n; i++)
```

```

10  {
11      scanf("%d", &tmp);
12      tmp = (tmp % k + k) % k;
13      e = 0;
14      for (int j = k - 1; j >= 0; j--)
15      {
16          if (f[j] == i - 1)
17          {
18              opt[++e][0] = f[j];
19              opt[e][1] = j;
20          }
21      }
22      for (int j = e; j >= 1; j--)
23      {
24          f[(opt[j][1] + tmp) % k] = opt[j][0] + 1;
25          f[(opt[j][1] - tmp + k) % k] = opt[j][0] + 1;
26      }
27  }
28  if (f[0] == n)
29      puts("Y");
30  else
31      puts("N");
32  return 0;
33 }

```

输入的 n 是不超过 100 的正整数,输入的 k 是不小于 2 且不超过 100 的正整数,输入的 tmp 的范围在 -10000 到 10000 之间,完成下面的判断题和单选题。

xxx 判断题

1. (1) 将第 5 行的“`memset(f, -1, sizeof(f));`”改为“`memset(f, 0, sizeof(f));`”,程序输出不变。()
2. (2) 将第 14 行的“`opt[++e][0] = f[j];`”改为“`opt[++e][0] = i - 1;`”,程序输出不变。()
3. (3) 将第 18 行的“`int j = e; j >= 1; j--`”改为“`int j = 1; j <= e; j++`”,程序输出不变。()
4. (4) 如果输入的 n 为 1000,程序会发生运行错误。()

xxx 单选题

5. (5) 如果输入的 n 为 4,输入的 4 个数分别为: 7, -6, -8, 4,当 k 分别为 5 和 7 时,输出分别为()。
 A. Y 和 Y B. Y 和 N C. N 和 Y D. N 和 N
6. (6) 可以将第 19 行和第 20 行中的“`opt[j][0] + 1`”替换为()。

A. $i-1$

B. i

C. $j-1$

D. j

2.

```
1#include <bits/stdc++.h>
2using namespace std;
3const int mod = 1000000007;
4int prime[1000010], tot, cnt[1000010], n;
5bool vis[1000010];
6long long ans = 1;
7void init()
8{
9    for (int i = 2; i <= n; i++)
10    {
11        if (!vis[i])
12            prime[++tot] = i;
13        for (int j = 1; j <= tot; j++)
14        {
15            if (prime[j] * i > n) break;
16            vis[prime[j] * i] = 1;
17            if (i % prime[j] == 0) break;
18        }
19    }
20}
21int main()
22{
23    scanf("%d", &n);
24    init();
25    for (int i = 1; i <= tot; i++)
26        for (int j = prime[i]; j <= n; j += prime[i])
27            for (int k = j; k % prime[i] == 0; k /= prime[i])
28                cnt[i]++;
29    for (int i = 1; i <= tot; i++)
30        ans = (ans * 1LL * (cnt[i] * 2 + 1) % mod) % mod;
31    printf("%lld\n", ans);
32    return 0;
33}
```

输入的 n 是不超过 10^6 的正整数,完成下面的判断题和单选题。

xxx 判断题

1. (1) 将第 9 行的“int i = 2;”改为“int i = 1;”,程序输出不变。()

2. (2) 将第 15 行删除,程序输出不变。()
3. (3) 将第 17 行删除,程序输出不变。()
4. (4) 将第 26 行的“`int j = prime[i]`”改为“`int j = 0`”,程序会出现死循环。()

xxx 单选题

5. (5) 如果输入的 n 为 2,输出为()。
- A. 1 B. 2 C. 3 D. 4
6. (6) 该代码中 `init()` 函数的时间复杂度为()。
- A. $O(\log n)$ B. $O(\sqrt{n})$ C. $O(n)$ D. $O(n \log n)$

3.

```

1#include < cstdio >
2int n, cnt, d[50005], f[50005], size[50005], head[50005];
3struct Edge
4{
5    int to, nxt;
6
7} edge[100005];
8void add(int x, int y)
9{
10    edge[++cnt].to = y;
11    edge[cnt].nxt = head[x];
12    head[x] = cnt;
13}
14void dfs1(int now)
15{
16    size[now] = 1;
17    for (int i = head[now]; i; i = edge[i].nxt)
18    {
19        int to = edge[i].to;
20        if (d[to])
21            continue;
22        d[to] = d[now] + 1;
23        dfs1(to);
24        size[now] += size[to];
25    }
26}
27void dfs(int now, int fa)
28{
29    f[now] = f[fa] + n - 2 * size[now];

```

```

30     for (int i = head[now]; i; i = edge[i].nxt)
31     {
32         int to = edge[i].to;
33         if (to == fa)
34             continue;
35         dfs(to, now);
36     }
37 }
38 int main()
39 {
40     scanf("%d", &n);
41     for (int x, y, i = 1; i < n; i++)
42     {
43         scanf("%d%d", &x, &y);
44         add(x, y); add(y, x);
45     }
46     d[1] = 1;
47     dfs1(1);
48     int maxn = 0, idx = 1;
49     for (int i = 1; i <= n; i++)
50         maxn += d[i];
51     maxn -= n;
52     f[1] = maxn;
53     for (int i = head[1]; i; i = edge[i].nxt)
54     {
55         int to = edge[i].to;
56         dfs(to, 1);
57     }
58     for (int i = 2; i <= n; i++)
59         if (f[i] < maxn)
60             maxn = f[i], idx = i;
61     printf("%d %d", idx, maxn);
62     return 0;
63 }

```

输入的 n 是不超过 1000 的正整数, x, y 是不超过 n 的正整数, 完成下面的判断题和单选题。

xxx 判断题

1. (1) 将第 44 行的“`add(x,y);add(y,x);`”改为“`add(y,x);add(x,y);`”, 程序输出不变。()
2. (2) 将第 20 行删除, 程序可能出现递归栈溢出风险。()
3. (3) 将第 34 行的“`continue;`”改为“`break;`”, 程序输出不变。()
4. (4) 将第 58 行的“`int i = 2;`”改为“`int i = 1;`”, 程序输出可能发生变化。()

xxx 单选题

5. (5) 如果输入数据如下所示:

4
1 2
2 3
3 4

则输出为()。

A. 1 2 B. 1 4 C. 2 3 D. 2 4

6. (6) (2分) 该代码的时间复杂度为()。

A. $O(n)$ B. $O(n\log n)$ C. $O(n^2)$ D. $O(n^3)$

三.完善程序 (单选题,每小题 3 分,共计 30 分)

1.

(黑白棋子) 一个棋盘被划分为 5 行 5 列一共 25 个格子,每个格子中都有一粒棋子,并且只有黑棋(标记为 H) 和白棋(标记为 J) 两种。如果一颗棋子在另一颗棋子的上下左右四个格子中的任一格里,我们说它们相连。现在要求选择一个包含 7 个相连的棋子区域,要求使得其中白棋比黑棋多。要求你编写一个程序求出满足要求的方案总数。

输入一个 5 行 5 列的矩阵,表示黑白棋子的位置情况。

输出格式:输出仅一个整数,表示方案总数。

输入样例: $HHHHH$

$JHJHJ$

$HHHHH$

$HJHHJ$

$HHHHH$

输出样例: 2

```
1#include <bits/stdc++.h>
2using namespace std;
3char str[7][7];
4int ans, sum, temp, c[8], d[8], flag[8], dir[4][2] = {1, 0, 0, 1,
-1, 0, 0, -1};
5void Jud(int p)
6{
7    flag[p] = 1, temp++;
8    for (int i = 0; i <= 3; i++)
9    {
10        int dx = c[p] + dir[i][0], dy = d[p] + dir[i][1];
11        for (int j = 1; j <= 7; j++)
12        {
```

```

13         if (      ① )
14             Jud(j);
15     }
16 }
17 }
18 void Sech(int x, int y)
19 {
20     if (y == 8)
21     {
22         temp = sum = 0;
23         memset(flag, 0, sizeof(flag));
24         for (int i = 1; i <= 7; i++)
25         {
26             if (      ② )
27                 sum++;
28         }
29         Jud(1);
30         if (      ③ )
31             ans++;
32         return;
33     }
34     if (x > 25)
35         return;
36     for (int i = x; i <= 25; i++)
37     {
38         ④ ;
39         ⑤ ;
40         Sech(i + 1, y + 1);
41     }
42 }
43 int main()
44 {
45     for (int i = 1; i <= 5; i++)
46         scanf("%s", str[i] + 1);
47     Sech(1, 1);
48     printf("%d\n", ans);
49     return 0;
50 }

```

1. (1) ①处应该填()

- A. $(c[j] == dx \&\& d[j] == dy) \&\& flag[j] == 0$
- B. $(c[j] == dx || d[j] == dy) || flag[j] == 0$

- C. $(c[j] == dx \&\& d[j] == dy) \parallel flag[j] == 0$
 D. $(c[j] == dx \parallel d[j] == dy) \&\& flag[j] == 0$

2. (2) ②处应该填 ()

- A. $str[i][i] == 'H'$ B. $str[i][i] == 'J'$
 C. $str[c[i]][d[i]] == 'H'$ D. $str[c[i]][d[i]] == 'J'$

3. (3) ③处应该填 ()

- A. $sum >= 4 \&\& temp == 7$ B. $sum >= 4 \&\& temp > 7$
 C. $sum > 4 \&\& temp == 7$ D. $sum > 4 \&\& temp > 7$

4. (4) ④处应该填 ()

- A. $c[y] = (i + 4)/4$ B. $c[y] = (i + 5)/5$
 C. $c[y] = (i + 5)/4$ D. $c[y] = (i + 4)/5$

5. (5) ⑤处应该填 ()

- A. $d[y] = i + ((i + 4)/5 - 1) * 5$ B. $d[y] = i + ((i + 4)/5 + 1) * 5$
 C. $d[y] = i - ((i + 4)/5 - 1) * 5$ D. $d[y] = i - ((i + 4)/5 + 1) * 5$

2.

(打牌) 三名同学在学习编程的休息时间 (编号 1, 2, 3) 打扑克, 每人一开始 n 张牌, 牌一共 m 种, 若干张相同的牌可以一起出。

一开始由第一个人出, 打出自己的牌里最小的牌。接下来, 以玩家 1, 2, 3, 1, 2, 3... 的顺序轮流打牌, 每人打出一组比上个人打出的牌大的, 自己能打出的最小的牌, 若没有则跳过。

牌的大小是这么决定的: 一组张数多的牌比张数少的牌大, 如果张数同样多, 那么点数大的牌比较大。例如: $(1, 1, 1) > (3, 3) > (2, 2) > (4) > (1)$ 。

若一轮中, 其余两人都无法打出牌, 则重新下次由打出最后一张牌的人开始打。谁最先打完所有的牌, 谁就赢了。请问最后谁会胜利呢? 输出胜者的编号。

对于所有数据, $n, m \leq 50$ 。

输入格式: 第 1 行输入 2 个正整数 n, m 。第 2 到 4 行, 每行输入 n 个数, 表示每个人一开始的牌。

输出格式: 输入共 1 行 1 个正整数, 表示胜者的编号。

输入样例:

```
10 3
1 3 3 1 3 3 1 2 3 3
3 2 1 2 2 3 3 1 1 2
2 2 1 2 3 1 2 3 3 1
```

输出样例:

```
3
```

```
1#include <bits/stdc++.h>
2#define int long long
```

```

3 using namespace std;
4 int n, m, a[5][55], x, last[5], sum[5], who = 1, tmp;
5 signed main()
6 {
7     cin >> n >> m;
8     sum[1] = sum[2] = sum[3] = n;
9     last[3] = 1;
10    for (int i = 1; i <= 3; i++)
11    {
12        for (int j = 1; j <= n; j++)
13        {
14            cin >> x;
15            ① ;
16        }
17    }
18    for (int i = 1;; i++)
19    {
20        tmp = 0;
21        who = (i - 1) % 3 + 1;
22        if (last[3] == who)
23            ② ;
24        for (int j = 0; j <= sum[who]; j++)
25        {
26            for (int k = 1; k <= m; k++)
27            {
28                if (          ③ )
29                    continue;
30                if (a[who][k] >= j + last[2])
31                {
32                    sum[who] - = j + last[2], tmp = 1;
33                    ④ ;
34                    ⑤ ;
35                    if (sum[who] == 0)
36                    {
37                        cout << who;
38                        return 0;
39                    }
40                    break;
41                }
42            }
43            if (tmp)
44                break;

```

```

45         }
46     }
47     return 0;
48 }

```

1. (1) ①处应该填 ()

- A. $a[i][x] = 1$ B. $a[i][x] += 1$ C. $a[x][i] = 1$ D. $a[x][i] += 1$

2. (2) ②处应该填 ()

- A. $last[1] = 0, last[2] = 0$ B. $last[1] = 0, last[2] = 1$
 C. $last[1] = 1, last[2] = 0$ D. $last[1] = 1, last[2] = 1$

3. (3) ③处应该填 ()

- A. $j == 0 \&\& k \leq last[1]$ B. $j == 1 \&\& k < last[1]$
 C. $j == 0 \&\& k < last[1]$ D. $j == 1 \&\& k \leq last[1]$

4. (4) ④处应该填 ()

- A. $last[1] = j, last[2] += who, last[3] = k$
 B. $last[1] = k, last[2] = j, last[3] = who$
 C. $last[1] = k, last[2] += j, last[3] = who$
 D. $last[1] = j, last[2] += k, last[3] = who$

5. (5) ⑤处应该填 ()

- A. $a[who][j] -= last[2]$ B. $a[who][k] -= last[2]$
 C. $a[who][j] += last[2]$ D. $a[who][k] += last[2]$